Binary search is one of the most powerful algorithms in a software engineer's toolkit. It's not just for finding a number in a sorted array; it's a framework for narrowing down a search space logarithmically.

1. The Fundamental Framework

The core idea is to maintain a "search space" and discard half of it in each step.

Key Elements to Remember:

- **Sorted Search Space:** The range must be monotonic (either increasing or decreasing).
- **The Midpoint Formula:** Use $low + (high - low)/2$ instead of $(low + high)/2$ to prevent integer overflow in languages like C++ or Java.
- **Termination Condition:** Pay close attention to whether you use `while (low <= high)` or `while (low < high)`. This depends on whether your search space is inclusive of `high`.

2. Core Pattern 1: Basic Binary Search (Exact Match)

Used when you need to find the exact index of a target value.

- **Logic:** If `nums[mid] == target`, return `mid`.
- **Example:** Find `7` in `[1, 3, 5, 7, 9]`.
- **Crucial Detail:** `low = mid + 1` and `high = mid - 1`.

3. Core Pattern 2: Binary Search on Duplicates (Leftmost/Rightmost)

Often, an array has multiple instances of the target, and you need the **first** or **last** occurrence.

- **Pattern (Leftmost):** When you find the target, don't stop. Set `high = mid` to keep looking in the left half to see if there's an earlier occurrence.
- **Pattern (Rightmost):** Set `low = mid + 1` (or a variation) to keep looking in the right half.
- **Example:** Finding the first `2` in `[1, 2, 2, 2, 3]`.

4. Advanced Pattern: Binary Search on "Answer"

This is the most common pattern in senior-level interviews. You aren't searching an array; you are searching a **range of possible answers**.

- **The Setup:** You have a range `[min_possible_answer, max_possible_answer]`.
- **The Predicate Function:** You create a helper function `isValid(k)` that returns `true` or `false` if a value `k` satisfies the problem constraints.
- **Example (Capacity/Allocation):** "Find the minimum capacity of a ship to transport all packages within D days."
  - `low`: Weight of the heaviest package.
  - `high`: Sum of all package weights.
  - Binary search for the smallest weight that makes `isValid(capacity)` true.

5. Advanced Pattern: Rotated Sorted Array

When a sorted array is shifted (e.g., `[4, 5, 6, 7, 0, 1, 2]`).

- **The Logic:** One half of the array (either `[low, mid]` or `[mid, high]`) **must** be sorted.
- **Strategy:** 1. Identify which half is sorted. 2. Check if the target lies within the range of that sorted half. 3. If yes, search there; otherwise, search the other half.

Pattern 1: The Standard Binary Search (Exact Match)

This is the most basic form of binary search. It is used when you are looking for the **exact position** of a specific value in a **sorted array**.

1. How to Recognize This Pattern?

You can identify this pattern if the problem has these characteristics:

- **Sorted Data:** The input array or list is already sorted (ascending or descending).
- **Unique Goal:** You are looking for a single specific value (e.g., "Find the index of number 7").
- **Direct Access:** You have random access to elements (like an array index).

3. The Approach (Step-by-Step)

1. **Initialize:** Start with two pointers: `low` at index 0 and `high` at the last index.
2. **Loop:** While `low` is less than or equal to `high`:
   - Calculate the middle index (`mid`).
   - **Check Match:** If `arr[mid] == target`, you found it! Return `mid`.
   - **Search Right:** If `target` is greater than `arr[mid]`, it must be in the right half. Update `low = mid + 1`.
   - **Search Left:** If `target` is smaller than `arr[mid]`, it must be in the left half. Update `high = mid - 1`.
3. **End:** If the loop finishes and you haven't returned, the target isn't in the array. Return `-1`.

```java
public int BinarySearch(int[] nums, int target) {
    int low = 0;
    int high = nums.Length - 1;

    // Use <= because the search space is [low, high] inclusive
    while (low <= high) {
        // Prevent integer overflow: low + (high - low) / 2
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) {
            return mid; // Target found
        }
        else if (nums[mid] < target) {
            low = mid + 1; // Look in the right half
        }
        else {
            high = mid - 1; // Look in the left half
        }
    }

    return -1; // Target not found
}
```

6. Most Important Things to Remember

- **Overflow Prevention:** Never use `(low + high) / 2`. In many languages (like Java), if `low` and `high` are very large, their sum will exceed the max integer value ($2^{31} - 1$), causing a crash or logic error. Always use `low + (high - low) / 2`.
- **The Exit Condition:** `while (low <= high)` is critical. If you use `low < high`, the loop will terminate before checking the very last element when the search space shrinks to size 1.
- **Efficiency:** The time complexity is $O(\log n)$. This means if you have 1 million elements, you only need about 20 comparisons.

Pattern 2: Binary Search for Boundaries (First/Last Occurrence)

This pattern is a step up from the standard search. It is used when the array contains **duplicate elements** and you need to find the specific index where the target "starts" or "ends."

1. How to Recognize This Pattern?

Look for these keywords in the problem description:

- **"Find the first occurrence..."** or **"Find the last occurrence..."**
- **"Find the range of a target..."** (requires finding both first and last).
- **"Find the lower bound / upper bound."**
- **Duplicates:** The problem explicitly mentions the array contains duplicates.

2. The Mental Model: "Don't Stop at the Match"

In Pattern 1, when `nums[mid] == target`, we return immediately. In Pattern 2, **we don't stop.**

- If we want the **First Occurrence**: Even if we find the target, there might be another one to the *left*. So, we record the current `mid` as a "potential candidate" and keep searching the left half.
- If we want the **Last Occurrence**: Record `mid` and keep searching the *right* half.

3. The Approach (Step-by-Step for First Occurrence)

1. **Initialize:** `low = 0`, `high = nums.Length - 1`, and `ans = -1`.
2. **Loop:** `while (low <= high)`:
    - Calculate `mid`.
    - **If** `nums[mid] == target`:
        - Update `ans = mid` (this is our current best answer).
        - **Crucial Step:** Set `high = mid - 1` (keep looking left).
    - **If** `nums[mid] < target`: `low = mid + 1`.
    - **If** `nums[mid] > target`: `high = mid - 1`.
3. **End:** Return `ans`.

```
public int FindFirst(int[] nums, int target) {
    int low = 0, high = nums.Length - 1;
    int firstIndex = -1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) {
            firstIndex = mid;   // Potential answer found
            high = mid - 1;     // Keep searching left for an earlier index
        }
        else if (nums[mid] < target) {
            low = mid + 1;
        }
        else {
            high = mid - 1;
        }
    }
    return firstIndex;
}
```

6. Most Important Things to Remember

- **The "Candidate" Variable:** Always store the `mid` in a temporary variable (`ans` or `res`) when you find a match. This ensures that if no other matches are found, you still have the correct index.
- **Binary Search on `bool` :** This pattern is essentially searching through a sequence of `false, false, true, true, true` (where "true" means "is this element $\geq$ target?"). You are looking for the first `true` .
- **Termination:** Unlike Pattern 1, where you might exit early, this pattern usually runs until `low > high` because it needs to "squeeze" the range to the very edge.

Pattern 3: Search in Rotated Sorted Array

This pattern is a classic favorite in technical interviews because it tests your ability to adapt the standard algorithm to a "broken" or shifted search space.

1. How to Recognize This Pattern?

You can quickly identify this pattern when the problem mentions:

- **"Rotated sorted array"**: An array like `[0, 1, 2, 4, 5]` becomes `[4, 5, 0, 1, 2]`.
- **"Unknown pivot"**: The array was shifted at some random point.
- **Efficient Search**: You are asked to find a target in $O(\log n)$ time even though the array isn't strictly increasing from start to finish.

2. The Mental Model: "One Half is Always Sorted"

The most important insight for this pattern is that **if you pick any middle element in a rotated sorted array, at least one of the two halves (left or right) MUST be sorted.**

- If `nums[low] <= nums[mid]`, then the **left half** is sorted.
- Otherwise, the **right half** must be sorted.

Once you identify the sorted half, you simply check if the target lies within that range. If it does, you search there; if not, you must search the other (possibly unsorted) half.

3. The Approach (Step-by-Step)

1. **Initialize:** `low = 0`, `high = nums.Length - 1`.
2. **Loop:** `while (low <= high)`:
    - Calculate `mid`.
    - **Direct Match:** If `nums[mid] == target`, return `mid`.
    - **Identify the Sorted Half:**
        - **Case A: Left half is sorted (`nums[low] <= nums[mid]`):**
            - Is the target between `nums[low]` and `nums[mid]`?
            - If yes: `high = mid - 1`.
            - If no: `low = mid + 1`.
        - **Case B: Right half is sorted:**
            - Is the target between `nums[mid]` and `nums[high]`?
            - If yes: `low = mid + 1`.
            - If no: `high = mid - 1`.
3. **End:** Return `-1`.

```
public int SearchInRotated(int[] nums, int target) {
    int low = 0, high = nums.Length - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (nums[mid] == target) return mid;

        // Check if the left half is sorted
        if (nums[low] <= nums[mid]) {
            // Target is in the sorted left half
            if (target >= nums[low] && target < nums[mid]) {
                high = mid - 1;
            } else {
                low = mid + 1;
            }
        }
        // Otherwise, the right half must be sorted
        else {
            // Target is in the sorted right half
            if (target > nums[mid] && target <= nums[high]) {
                low = mid + 1;
            } else {
                high = mid - 1;
            }
        }
    }
    return -1;
}
```

6. Most Important Things to Remember

- **The Equality Check:** Use `nums[low] <= nums[mid]` (with the `<=`) to handle cases where the search space has only two elements.
- **The "If/Else" logic:** You must first find **which** side is sorted before you try to check if the target is in that side. You cannot check the target's range on an unsorted side.
- **Duplicate Handling:** Note that this specific implementation works for **distinct** elements. If the array has duplicates (like `[1, 0, 1, 1, 1]`), you might need a small modification to handle the case where `nums[low] == nums[mid] == nums[high]`.

Pattern 4: Binary Search on "Answer Space"

This is widely considered the most "senior-level" binary search pattern. Unlike previous patterns where you search for an index in an array, here you are searching for a **numeric value** within a range of possible answers.

1. How to Recognize This Pattern?

This pattern is often hidden behind optimization problems. Look for:

- **Keywords:** "Find the **minimum** value of X such that...", "Find the **maximum** distance...", "Minimize the maximum...".
- **A Range of Possibilities:** You can clearly define a "lowest possible answer" and a "highest possible answer."
- **Monotonicity (The "Yes/No" Rule):** If a value k works, then every value larger than k also works (or vice versa). This creates a sequence like `[No, No, No, Yes, Yes, Yes]`.
- **The "Hard" Part:** Calculating the answer directly is difficult, but **checking** if a specific answer is valid is relatively easy (usually $O(N)$).

2. The Mental Model: "The Guessing Game"

Instead of trying to calculate the perfect answer, you **guess** a middle value from your range and ask a helper function: *"Is this guess valid/possible?"*

- If the guess is **too small** to be valid, you discard the lower half.
- If the guess **is valid**, you save it as a candidate and try to find an even "better" (smaller/larger) valid answer in the other half.

3. The Approach (Step-by-Step)

1. **Define the Search Space:**
   - `low`: The absolute minimum possible answer (often `1`, `min(arr)`, or `0`).
   - `high`: The absolute maximum possible answer (often `sum(arr)` or `max(arr)`).
2. **The Predicate Function (`isPossible`):**
   - Write a helper function that takes your "guess" and returns `true` or `false` based on the problem constraints.
3. **The Binary Search:**
   - While `low <= high`:
     - `mid` = your guess.
     - If `isPossible(mid)` is true:
       - Record `mid` as `ans`.
       - Move towards the "better" side (if minimizing, `high = mid - 1`; if maximizing, `low = mid + 1`).
     - If `false`:
       - Move to the other side to find a valid range.

```csharp
public int MinimizeMaximum(int[] weights, int days) {
    int low = weights.Max(); // Min possible: heaviest single item
    int high = weights.Sum(); // Max possible: all items in one go
    int ans = high;

    while (low <= high) {
        int mid = low + (high - low) / 2;

        if (IsPossible(weights, days, mid)) {
            ans = mid;        // Current guess works, save it
            high = mid - 1;   // Try to find a smaller "minimum"
        } else {
            low = mid + 1;    // Guess was too small, increase it
        }
    }
    return ans;
}

// Helper function: Can we ship weights within 'days' using 'capacity'?
private bool IsPossible(int[] weights, int days, int capacity) {
    int currentDays = 1, currentLoad = 0;
    foreach (int w in weights) {
        if (currentLoad + w > capacity) {
            currentDays++;
            currentLoad = 0;
        }
        currentLoad += w;
    }
    return currentDays <= days;
}
```

6. Most Important Things to Remember

- **Boundary Selection:** Choosing the correct `low` and `high` is 50% of the work. If your range is wrong, your answer will be wrong.
- **Complexity:** The complexity is $O(N \cdot \log(\text{range}))$, where N is the cost of the `IsPossible` check.
- **The "Threshold":** This pattern transforms a "find the best" problem into a "is this value possible" problem.